CascadeVerify: Static Detection of Retry Amplification and Timeout Chain Violations in Microservice Configurations via Bounded Model Checking and MaxSAT Repair

CascadeVerify Team

2026

Abstract

Modern microservice deployments compose retry policies, timeout budgets, and circuit-breaker behavior across independently managed services. Even when each local configuration appears reasonable, their composition can create retry amplification and timeout-chain failures that are not visible in any single manifest. We present CascadeVerify, a formalization-first project for reasoning about these risks through a *Retry-Timeout Interaction Graph* (RTIG), bounded model checking, minimal-failure-set enumeration, and MaxSAT-style repair.

This paper deliberately centers the strongest reproducible evidence currently in the repository: the formal model and proofs, a working Python reference implementation, and two rerunnable evaluation scripts. On a transparent suite of 14 hand-authored scenarios, the reference implementation agrees with the scenario labels in all 14 cases, with average verification time 1.8 ms and average repair time 3.6 ms; 13 of the 14 repaired scenarios re-check as safe. On generated topologies up to 100 services, the same implementation completes in 1.37 s for a chain, 0.49 s for a tree, and 3.56 s for a mesh. A separate "deep cascade" profiler projects larger-depth costs, but we report those numbers explicitly as model-based projections rather than measured tool runs. The repository also contains a substantially larger Rust CLI prototype, which we treat as engineering scaffolding rather than the core validated story.

1 Introduction

Microservice architectures have become the dominant deployment model for large-scale web services [15]. A critical consequence of this decomposition is that resilience mechanisms—retry policies, timeout budgets, circuit breakers, and rate limits—are configured independently by each service team, yet their cross-service interactions create an emergent failure surface that is invisible to any single team's configuration review.

The most frequently cited cascade mechanisms in production post-mortems are *retry amplification* and *timeout chain violations*:

- **Retry amplification:** When service A retries r_1 times against B, and B retries r_2 times against C, the effective load on C is $(1 + r_1)(1 + r_2)$ times baseline. Along a chain of n edges, the worst-case amplification is $\prod_{i=1}^{n}(1 + r_i)$—exponential in path length for $r_i > 0$.
- **Timeout chain violations:** The end-to-end latency under retry expansion is $\sum_{i=1}^{n} t_i \cdot (1 + r_i)$ where t_i is the per-try timeout on edge i. If this exceeds the upstream caller's deadline, partial degradation becomes total unavailability.

These interactions caused the 4-hour AWS S3 outage in February 2017 [1], the Google Cloud multi-region networking outage in June 2019 [2], and the global Cloudflare outage in July 2019 [3].

Positioning. Existing resilience-analysis approaches typically fall into three families: per-resource configuration validators, manual formal specifications, and runtime fault-injection systems. CascadeVerify aims at a different point in this space: extract a lightweight formal model directly from service configuration structure and reason statically about retry/timeout composition. The current repository most strongly validates that formalization and an executable reference implementation; it does *not* yet justify broad quantitative head-to-head claims against external tools.

Contributions. We make the following contributions:

1. We formalize the *Retry-Timeout Interaction Graph* (RTIG), a directed graph annotated with per-edge retry counts and timeout budgets, capturing the two cascade mechanisms emphasized

throughout this repository (§2).

2. We prove that bounded cascade reachability in the RTIG model is **NP-complete**, that CB-free RTIGs are monotone, and that this yields a concrete completeness bound for bounded model checking (§§2–3).
3. We encode RTIG analysis as QF_LIA bounded model checking with MUS/MARCO enumeration of minimal cascading failure sets and a weighted partial MaxSAT repair formulation (§§4–5).
4. We provide a compact, executable Python reference implementation that realizes the formal model and can be rerun directly from the repository to reproduce the scenario-suite and scaling artifacts (§6, §7).
5. We document the larger Rust workspace as a promising engineering prototype, but not as the primary validated artifact of this paper (§6).

2 Formal Model

Definition 1 (RTIG)**.** *A* Retry-Timeout Interaction Graph *is a tuple $G = (V, E, \kappa, \rho)$ where V is the set of services, $E \subseteq V \times V$ is the dependency relation, $\kappa : V \to \mathbb{N}$ assigns capacity, and $\rho : E \to \mathbb{N} \times \mathbb{N}$ maps each edge (u, v) to a pair $(\mathit{retries}(u, v), \mathit{timeout}(u, v))$.*

Definition 2 (Load Propagation)**.** *The effective load on service v at discrete time step t under failure set $F \subseteq V$ is:*

$$L(v,t,F) = \begin{cases} 0 & \text{if } v \in F, \\ \mathit{base}(v) + \sum_{u \in \mathrm{pred}(v)} L(u, t{-}1, F) \cdot (1 + r_{u,v}) & \text{otherwise,} \end{cases}$$

where $r_{u,v} = \rho(u, v).\mathit{retries}$.

Definition 3 (Cascade)**.** *A* cascade *occurs under failure set F if there exist a service $v^* \in V$ (the entry point) and time step $t \le d^*$ such that $L(v^*, t, F) > \kappa(v^*)$.*

Proposition 1 (NP-Completeness)**.** *The bounded cascade reachability problem—"does there exist $F \subseteq V$ with $|F| \le k$ inducing a cascade?"—is* ***NP-complete****.*

Proof sketch. Membership in NP: guess F, verify cascade via load propagation in $O(|V|^2 \cdot d^*)$ time. NP-hardness: reduction from Subset-Sum. Given $(a_1, \ldots, a_n, T)$, construct an RTIG with n services each having capacity a_i and a single entry point whose load under failure set F equals $\sum_{i \in F} a_i$. The entry point cascades iff $\sum_{i \in F} a_i > T$. □ □

3 Monotonicity and Pruning

Theorem 2 (Monotonicity of CB-free RTIGs)**.** *For any RTIG G without circuit breakers, if failure set F induces a cascade, then every superset $F' \supseteq F$ also induces a cascade.*

Proof. By structural induction on the unrolling depth t. The base case $t = 0$ is immediate: additional failures do not decrease baseline load. For the inductive step, assume $L(v, t, F) \le L(v, t, F')$ for all $v \notin F'$ and all $t' < t$. Then:

$$L(v,t,F') = \mathit{base}(v) + \sum_{u \in \mathrm{pred}(v)} L(u, t{-}1, F') \cdot (1 + r_{u,v})$$

By the inductive hypothesis, each $L(u, t{-}1, F') \ge L(u, t{-}1, F)$ (services in $F' \setminus F$ contribute zero load but no longer absorb incoming requests, increasing load on their successors' remaining predecessors). Since multiplication by $(1 + r_{u,v}) \ge 1$ preserves the inequality, $L(v, t, F') \ge L(v, t, F)$.

If F induces a cascade ($L(v^*, t, F) > \kappa(v^*)$ for some t), then $L(v^*, t, F') \ge L(v^*, t, F) > \kappa(v^*)$, so F' also induces a cascade. □ □

Corollary 3 (Completeness Bound)**.** *For a monotone RTIG, $d^* = \mathrm{diam}(G) \times \max_e r_e$ suffices for BMC completeness.*

Proof. Let $D = \mathrm{diam}(G)$ and $r_{\max} = \max_e r_e$. By the load propagation recurrence (Definition 2), a cascade originating at any service v propagates through at most D hops to reach the entry point v^*. At each hop, the retry mechanism can delay propagation by up to $r_{\max}$ additional time steps (each retry attempt is modeled as a separate step). Therefore the worst-case propagation delay from any initial failure to a load-threshold violation at v^* is at most $D \cdot r_{\max}$ steps.

Formally, suppose failure set F induces a cascade, i.e., there exists some t_0 with $L(v^*, t_0, F) > \kappa(v^*)$. Because the load function $L(v, t, F)$ is monotonically non-decreasing in t for fixed F (by Theorem 2 and the non-negative, additive structure of the recurrence), if no threshold violation occurs by step $d^* = D \cdot r_{\max}$, then $L(v^*, t, F) \le \kappa(v^*)$ for all t. Equivalently, $t_0 \le d^*$ whenever a cascade exists, so unrolling the BMC formula to depth d^* is complete: any satisfiable cascade witness is found within this bound. □ □

The monotonicity theorem enables *antichain pruning*: when enumerating minimal failure sets, we can skip (a) supersets of known cascading sets and (b) subsets of known safe sets.

Circuit-breaker abstraction. Circuit breakers break monotonicity: protective tripping can *reduce* load on downstream services, so a superset of failures may *not* cascade when a subset does. We recover monotonicity via a sound over-approximation. Each CB is modelled as a three-element lattice *Closed* $\leq$ *HalfOpen* $\leq$ *Open*, and we abstract every CB that *could* trip to the absorbing state *Open*. An open CB contributes *backpressure* load $\beta(e) \geq 1$ to the *caller* (fast-fail overhead) while zeroing traffic to the callee. Because backpressure is non-negative and more failures can only force *more* CBs open, the abstract load function $\hat{L}(v, t, F)$ is monotonically non-decreasing in F. Formally, $\hat{L}(v, t, F) \geq L(v, t, F)$ for every v, t, F (the abstraction never under-counts load), so every concrete cascade is detected. The cost is a conservative increase in false positives—cascades that a real CB would prevent—but no false negatives. This enables the existing antichain pruning (§3) and MARCO loop (the MARCO loop) to operate unchanged on CB-inclusive topologies.

4 BMC Encoding

We encode the RTIG as a bounded model checking problem in the quantifier-free linear integer arithmetic (QF_LIA) fragment. For each service $v \in V$ and time step $t \in [0, d^*]$, we introduce variables:

- $\mathit{load}[v, t] \in \mathbb{Z}_{\geq 0}$: effective load at step t
- $\mathit{failed}[v] \in \{0, 1\}$: initial failure indicator
- $\mathit{state}[v, t] \in \{0, 1, 2\}$: healthy/degraded/unavailable

The BMC formula Φ_{BMC} is the conjunction:

$$\Phi_{\text{init}} : \text{failed services start unavailable} \tag{1}$$

$$\Phi_{\text{trans}} : \text{load propagation with retry amplification} \tag{2}$$

$$\Phi_{\text{casc}} : \exists t.\ \mathit{load}[v^*, t] > \kappa(v^*) \tag{3}$$

$$\Phi_{\text{bound}} : \sum_{v} \mathit{failed}[v] \leq k \tag{4}$$

Optimizations.

- **Symmetry breaking:** When services share identical resilience policies and graph structure, we impose a lexicographic ordering on their failure variables, reducing the search space by $n!/|\text{Aut}(G)|$. *(Speedup: empirical validation pending.)*
- **Cone-of-influence:** For a given entry point v^*, only services in the reverse-reachable set of v^* can contribute to a cascade. We eliminate all other variables. *(Reduction: empirical validation pending.)*
- **Incremental solving:** When checking increasing failure budgets $k = 1, 2, \ldots$, we reuse learned clauses from $k - 1$ via solver push/pop. *(Speedup: empirical validation pending.)*

4.1 MUS Enumeration

We adapt the MARCO algorithm [17] for minimal unsatisfiable subset enumeration to discover all minimal failure sets. Given the negated cascade-freedom formula, each MUS corresponds to a minimal set of failure assumptions that, when removed, make the formula satisfiable—i.e., a minimal failure set inducing a cascade.

Theorem 4 (MinUnsat Correspondence)**.** *MinUnsat cores of the negated cascade-freedom formula correspond exactly to minimal failure sets inducing cascades.*

Proof. Let Φ_{safe} denote the cascade-freedom formula: the conjunction asserting $L(v^*, t, F) \leq \kappa(v^*)$ for all $t \leq d^*$ and entry point v^*. For each service $s_i \in V$, introduce an assumption literal a_i that controls the failure of s_i: when a_i is asserted, service s_i is assumed to have failed (i.e., $\mathit{failed}[s_i] = 1$). The augmented formula is $\Psi = \Phi_{\text{safe}} \wedge \bigwedge_i a_i$.

(Forward direction.) Let $M = \{a_{i_1}, \ldots, a_{i_m}\}$ be a minimal unsatisfiable subset (MUS) of the assumption literals with respect to Φ_{safe}. By definition, $\Phi_{\text{safe}} \wedge \bigwedge_{a \in M} a$ is unsatisfiable, meaning that the simultaneous failure of the services $F_M = \{s_{i_1}, \ldots, s_{i_m}\}$ makes cascade-freedom impossible—i.e., F_M induces a cascade. Minimality of M as an unsatisfiable subset implies that removing any single a_{i_j} makes the conjunction satisfiable: the failure set $F_M \setminus \{s_{i_j}\}$ does *not* necessarily induce a cascade. Hence F_M is a *minimal* cascading failure set.

(Reverse direction.) Let $F = \{s_{i_1}, \ldots, s_{i_m}\}$ be a minimal cascading failure set. Then $\Phi_{\text{safe}} \wedge \bigwedge_{j=1}^{m} a_{i_j}$ is unsatisfiable (cascade-freedom fails under F). Suppose for contradiction that the corresponding assumption set $M_F = \{a_{i_1}, \ldots, a_{i_m}\}$ is not a MUS—then some proper subset $M' \subsetneq M_F$ already makes Φ_{safe} unsatisfiable, meaning the corresponding proper subset $F' \subsetneq F$ also induces a cascade. This contradicts the minimality of F. Therefore M_F is a MUS.

The two directions establish a bijection $F \leftrightarrow M_F$ between minimal cascading failure sets and MUSes of assumption literals with respect to the cascade-freedom formula. □ □

Monotonicity-aware pruning integrates with MARCO by maintaining antichains of known cas-

cading and safe sets, skipping supersets and subsets respectively.

5 MaxSAT Repair Synthesis

Given the set of discovered cascade scenarios, we synthesize repairs via weighted partial MaxSAT:

- **Hard clauses:** For each minimal failure set F_i, assert that the repaired configuration does not cascade under F_i.
- **Soft clauses:** For each parameter p_j, assert $p_j = p_j^{\text{orig}}$ with weight proportional to operational impact.

The MaxSAT solver maximizes the total weight of satisfied soft clauses subject to all hard clauses, producing repairs that minimize total parameter deviation.

Theorem 5 (Repair Complexity)**.** *Optimal repair synthesis—finding a minimum-cost parameter modification guaranteeing cascade-freedom—is Σ_2^P-complete.*

Proof sketch. The $\forall\exists$ quantifier alternation arises from universally quantifying over failure sets ($\forall F, |F| \leq k$) while existentially quantifying over load propagation witnesses ($\exists t.\ L(v^*, t, F) \leq \kappa(v^*)$). Hardness follows from the known Σ_2^P-completeness of the min-cost repair problem for monotone Boolean formulas. Membership uses the NP oracle for cascade verification. □ □

We enumerate the Pareto frontier of repair options by iteratively blocking previously found optima, typically producing 2–5 distinct repair strategies.

6 Implementation

The repository currently contains two distinct implementation artifacts.

Executable reference implementation. The paper's quantitative evidence is grounded in `benchmarks/empirical_evaluation.py`, a compact Python reference implementation of the RTIG load-propagation recurrence, minimal-failure-set enumeration, anti-pattern detection, and MaxSAT-style repair loop. This script writes `benchmarks/empirical_results.json`. A second script, `benchmarks/deep_cascade_profiler.py`, writes `benchmarks/deep_cascade_results.json`; unlike the empirical evaluator, it generates projected phase times through an explicit simulation function (`_simulate_phase_times`) rather than executing the verifier.

Table 1: Repository artifacts used in the recovered paper story.

Artifact	Role	Validation status
Python reference implementation	RTIG propagation, failure-set enumeration, repair, small-scale timing	rerun successfully
`empirical_results.json`	executed scenario suite + measured scaling to 100 services	regenerated
`deep_cascade_results.json`	model-based deep-topology projections	regenerated, explicitly simulated
Rust workspace	parser / CLI / reporting prototype	build passes; tests and example invocation currently unstable

Rust workspace prototype. The directory `implementation/` contains a 10-crate Rust workspace. A simple line count over `implementation/crates/**/*.rs` yields 79 Rust source files and 62,179 lines. On a fresh rerun for this recovery pass, `cargo build -workspace -quiet` succeeded, but `cargo test -workspace -quiet` failed because several tests and APIs have drifted out of sync, and a sample invocation of `./target/debug/cascade-verify check ../examples/retry-amplification` panicked with a CLI flag type mismatch. We therefore treat the Rust workspace as engineering scaffolding and use the Python reference implementation as the validated core artifact.

7 Evaluation

We report only evidence that can be reproduced directly from repository code. The evaluation has three parts: (1) an executable scenario-suite validation of the Python reference implementation, (2) measured scaling on generated chain/tree/mesh topologies up to 100 services, and (3) a clearly labeled model-based deep-topology profiler.

7.1 Executable Reference Validation

The script `benchmarks/empirical_evaluation.py` defines 14 transparent scenarios: 9 intended to cascade and 5 intended to remain safe. These include small didactic topologies (retry amplification, timeout chains, fan-in storms), service-mesh-flavored examples

Table 2: Rerun of the executable Python reference implementation on the 14-scenario suite.

Metric	Value
Scenarios	14 (9 cascade, 5 safe)
Agreement with scenario labels	14 / 14
Average verification time	1.8 ms
Average repair time	3.6 ms
Cascade scenarios with sound repair	13/14 (92.9%)
Average parameter changes on cascade scenarios	1.6

Table 3: Measured total runtime (ms) of the Python reference implementation on generated topologies.

Topology	10 svc	30 svc	50 svc	100 svc
Chain	1.4	44.1	185.8	1365.8
Tree	0.5	10.5	52.0	491.7
Mesh	0.3	13.3	161.2	3561.6

(`bookinfo_istio`), and a denser 20-service mesh. Because the scenarios and their labels are hand-authored in that same script, the appropriate interpretation is *agreement with a transparent scenario suite*, not external precision/recall on an audited corpus.

The negative result is also important: the dense `mesh_20_services` scenario remains unsound after the current repair loop, so the repair story is not yet "solved." This is precisely why the paper emphasizes the reference implementation as a faithful, inspectable model rather than a polished production verifier.

7.2 Measured Scaling on Generated Topologies

The same script generates synthetic chain, tree, and mesh graphs at 10, 30, 50, and 100 services. For each graph it runs a Tier-1 anti-pattern pass and a Tier-2 single-failure exploration loop. These measurements are genuine runs of the Python reference implementation.

At 100 services, the chain case completes in 1.37 s, the tree in 0.49 s, and the mesh in 3.56 s. These are still generated graphs rather than production manifests, but they establish an honest, executable scalability claim: the reference model runs comfortably on low-hundreds-of-service synthetic configurations, with dense connectivity becoming the dominant cost driver.

Table 4: Selected projected results from the deep-topology profiler.

Configuration	Time (s)	Mem (MB)	Bottleneck
Chain, $w = 2$, $d = 50$	18.76	75.0	constraint generation
Tree, $w = 2$, $d = 50$	41.14	148.5	constraint generation
Mesh, $w = 2$, $d = 50$	235.33	300.0	SMT solving
Mesh, $w = 5$, $d = 50$	755.05	750.0	SMT solving
Mesh, $w = 10$, $d = 30$	576.72	780.0	SMT solving
Mesh, $w = 10$, $d = 50$	2067.26	1500.0	SMT solving

7.3 Model-Based Deep-Topology Profiler

The repository also contains `benchmarks/deep_cascade_profiler.py`, which sweeps depths 5–50, widths 2/5/10, and chain/tree/mesh topology families. It is important to state exactly what this script does: it generates topology metadata and then calls `_simulate_phase_times` to project verification cost. The resulting numbers are useful as a *projection* of where cost may arise, but they are not measured end-to-end verifier runtimes.

Two qualitative conclusions survive this caveat. First, the profiler consistently identifies constraint generation as the dominant phase for chain and tree families, and SMT solving as the dominant phase for mesh families. Second, the deepest and widest meshes are the first projected configurations to hit the 600 s timeout threshold. The polynomial fits reported in `deep_cascade_results.json` are best understood as properties of the profiler's own formulas, not of measured tool execution.

7.4 Threats to Validity

Scenario-suite bias. The 14-scenario suite is explicit and rerunnable, but it is small and hand-authored. Agreement on that suite is evidence that the reference implementation behaves consistently on transparent examples, not evidence of external generalization.

Repair evaluation is internal. The repair loop is evaluated within the same reference implementation used to generate scenarios and to re-check them. The 13/14 soundness figure is still useful, but it should be read as internal consistency evidence rather than a competitive benchmark.

Deep-topology data are projected. The deep-cascade profiler is model-based. We report it because

it is executable and informative, but every projected timing statement in this paper is labeled as such.

Rust prototype mismatch. The Rust workspace build succeeds, yet the current test suite and one example CLI invocation fail. This is why the paper's validated story is anchored in the Python reference implementation rather than the CLI.

8 Related Work

Our recovered story positions CascadeVerify relative to prior work qualitatively, not through unrun benchmark tables.

Configuration validators. Tools such as kubescore, OPA Gatekeeper, Polaris, and `istioctl analyze` focus on schema correctness, best practices, and policy conformance of individual resources or local relationships. They are the closest operational neighbors to CascadeVerify because they run before deployment, but this paper does not claim quantitative superiority over them; such claims would require an audited shared corpus and actual head-to-head runs.

Manual formal methods. TLA+ [5, 7] and P [6] demonstrate that distributed systems benefit from explicit formal modeling. CascadeVerify shares that motivation, but targets a lighter-weight workflow in which a model is extracted from service configuration rather than written from scratch.

Runtime fault injection. LDFI [4] and chaos-engineering systems such as Chaos Monkey [12], Chaos Mesh [14], and LitmusChaos [13] explore failures dynamically after deployment. They are complementary to CascadeVerify's static, pre-deployment perspective.

9 Discussion and Limitations

Circuit breakers. The monotonicity theorem (Theorem 2) holds only for CB-free networks. We address this via a sound over-approximation (§3): each CB that could trip is abstracted to an absorbing *permanently-open* state, redirecting load as caller-side backpressure. This recovers monotonicity at the cost of potential false positives.

Static snapshots. The formal model analyzes static configuration snapshots. Runtime adaptation, autoscaling, transient traffic bursts, and per-request routing decisions are not fully captured.

Reference-model scope. The strongest executable evidence currently comes from the Python reference implementation on transparent synthetic scenarios and generated topologies. The next step is to move from "scenario-suite agreement" to externally audited manifest corpora.

Engineering gap. The repository already contains a much larger Rust implementation, but its tests and example CLI invocation currently lag behind the reference model. Bringing those two artifacts into alignment is a concrete engineering milestone, not a minor cleanup task.

10 Conclusion

CascadeVerify's truthful current story is a formalization-first one. The repository contains a clear RTIG model, proofs for the main structural properties that make analysis possible, and a working Python reference implementation that can be rerun to reproduce transparent scenario-suite and small-scale scalability results. That is already a meaningful contribution: it turns the intuition that "retries and timeouts compose badly" into a concrete, inspectable formal artifact.

The right next milestone is not another speculative comparison table. It is to align the Rust prototype with the reference model, build an audited benchmark corpus from real manifests, and then rerun end-to-end experiments with the same level of disclosure used in this recovered version of the paper.

References

[1] Amazon Web Services. Summary of the Amazon S3 Service Disruption in the Northern Virginia (US-EAST-1) Region. `https://aws.amazon.com/message/41926/`, 2017.

[2] Google Cloud. An update on Sunday's service disruption. `https://cloud.google.com/blog/topics/inside-google-cloud/an-update-on-sundays-service-disruption`, 2019.

[3] Cloudflare. Details of the Cloudflare outage on July 2, 2019. `https://blog.cloudflare.com/details-of-the-cloudflare-outage-on-july-2-2019/`, 2019.

[4] P. Alvaro, J. Rosen, and J. M. Hellerstein. Lineage-driven fault injection. In *Proceedings of ACM SIGMOD*, pages 331–346, 2015.

[5] L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[6] A. Desai, V. Gupta, E. Jackson, S. Qadeer, S. Rajamani, and D. Zufferey. P: Safe asynchronous event-driven programming. In *Proceedings of PLDI*, pages 321–332, 2013.

[7] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff. How Amazon web services uses formal methods. *Communications of the ACM*, 58(4):66–73, 2015.

[8] kube-score. Kubernetes object analysis with recommendations for improved reliability and security. `https://github.com/zegl/kube-score`, 2023.

[9] Open Policy Agent. OPA Gatekeeper: Policy controller for Kubernetes. `https://github.com/open-policy-agent/gatekeeper`, 2023.

[10] Istio. istioctl analyze. `https://istio.io/latest/docs/reference/commands/istioctl/#istioctl-analyze`, 2023.

[11] Fairwinds. Polaris: Validation of best practices in Kubernetes clusters. `https://github.com/FairwindsOps/polaris`, 2023.

[12] Netflix. Chaos Monkey. `https://github.com/Netflix/chaosmonkey`, 2016.

[13] LitmusChaos. Litmus: Cloud-native chaos engineering. `https://litmuschaos.io/`, 2020.

[14] Chaos Mesh. Chaos Mesh: A powerful chaos engineering platform for Kubernetes. `https://chaos-mesh.org/`, 2021.

[15] S. Newman. *Building Microservices*. O'Reilly Media, 2015.

[16] M. T. Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2nd edition, 2018.

[17] M. H. Liffiton, A. Malik, and M. H. Sakallah. Enumerating infeasibility: Finding multiple MUSes quickly. In *Proceedings of CPAIOR*, pages 160–175, 2016.

[18] Google Cloud. Online Boutique: Cloud-native microservices demo application. `https://github.com/GoogleCloudPlatform/microservices-demo`, 2023.

[19] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels. Dynamo: Amazon's highly available key-value store. In *Proceedings of SOSP*, pages 205–220, 2007.

[20] StackRox (Red Hat). KubeLinter: Static analysis for Kubernetes. `https://github.com/stackrox/kube-linter`, 2023.

[21] Datree. Datree: Prevent Kubernetes misconfigurations from reaching production. `https://github.com/datreeio/datree`, 2023.

[22] Gremlin. Gremlin: Chaos engineering platform. `https://www.gremlin.com/`, 2023.

[23] Facebook Engineering. More details about the October 4 outage. `https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/`, 2021.

[24] E. M. Clarke, O. Grumberg, D. Kroening, D. Peled, and H. Veith. *Model Checking*. MIT Press, 2nd edition, 2018.

[25] C. M. Li and F. Manyà. MaxSAT, hard and soft constraints. In A. Biere, M. Heule, H. van Maaren, and T. Walsh, editors, *Handbook of Satisfiability*, chapter 19. IOS Press, 2nd edition, 2021.

[26] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal. Chaos engineering. *IEEE Software*, 33(3):35–41, 2016.

[27] W. Morgan and O. Gould. Linkerd: Ultralight, security-first service mesh for Kubernetes. `https://linkerd.io/`, 2021.

[28] Linkerd. Configuring retries. `https://linkerd.io/2/tasks/configuring-retries/`, 2023.

[29] Envoy Project. Envoy proxy: Cloud-native high-performance edge/middle/service proxy. `https://www.envoyproxy.io/`, 2023.

[30] Netflix. Zuul: Gateway service providing routing, monitoring, resiliency, and security. `https://github.com/Netflix/zuul`, 2023.

[31] Amazon Web Services. AWS App Mesh: Application-level networking for all your services. https://aws.amazon.com/app-mesh/, 2023.

[32] Kubernetes. Configure liveness, readiness and startup probes. https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/, 2023.

[33] C. Luo, L. Xu, D. Wang, and Z. Li. Characterizing microservice dependency and performance: Alibaba trace analysis. In *Proceedings of ACM SoCC*, pages 412–426, 2021.

[34] M. Tirmazi, A. Barker, N. Deng, M. E. Haque, Z. G. Qin, S. Hand, M. Harchol-Balter, and J. Wilkes. Borg: The next generation. In *Proceedings of EuroSys*, pages 30:1–30:14, 2020.

[35] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cober, J. Flores, and R. Bianchini. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *Proceedings of USENIX ATC*, pages 205–218, 2020.

[36] Semgrep. Semgrep: Lightweight static analysis for many languages. https://semgrep.dev/, 2023.

[37] Cloud Native Computing Foundation. CNCF Kubernetes conformance. https://www.cncf.io/certification/software-conformance/, 2023.

[38] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.